

M1 Mathématiques – Informatique pour Mathématiciens

Séance 2 : Récursivité, Diviser pour Régner
et Introduction à la POO

Université – Année 2026–2027

Table des matières

1	Préambule : L'IA, partenaire de réflexion	2
2	Objectifs de la séance	2
3	Introduction à la Programmation Orientée Objet (POO)	2
3.1	Pourquoi la POO pour un mathématicien ?	2
3.2	Syntaxe de base d'une classe en Python	3
3.3	Avantages de la POO	4
4	Récursivité : principes et mécanismes	4
4.1	Pièges classiques	5
5	Paradigme « Diviser pour Régner »	5
6	Algorithmes à implémenter	6
6.1	Exponentiation rapide	6
6.2	Multiplication de Karatsuba	7
6.3	Tri fusion (Merge Sort)	8
7	Exercices	9
7.1	Exercices sur la récursivité	9
7.2	Exercices sur la POO	9
7.3	Exercices sur Diviser pour Régner	9
8	Analyse de complexité et mesures	10
8.1	Application du Master Theorem	10
8.2	Mesures empiriques	10
9	Livrable étudiant	10

1 Préambule : L'IA, partenaire de réflexion

Comme évoqué lors de la première séance, l'usage des assistants d'IA générative est **autorisé** dans ce cours. Toutefois, je vous rappelle le rôle central du mathématicien :

- **Analyser la structure mathématique** d'un problème (relation de récurrence, complexité asymptotique).
- **Concevoir des classes et des interfaces cohérentes** qui reflètent cette structure.
- **Vérifier la correction** des algorithmes, notamment lorsque les cas de base et les appels récursifs s'enchaînent.

Un assistant peut vous aider à écrire du code, mais il ne saura pas **garantir** que votre implémentation de Karatsuba fonctionne pour des entiers de 1000 chiffres, ni que votre analyse de complexité via le Master Theorem est correcte. Cette responsabilité vous incombe.

Ressources

Ressources recommandées pour cette séance :

- Cormen, Leiserson, Rivest, Stein – *Introduction to Algorithms* (4e édition, 2022), chapitres 4 et 30. Ouvrage de référence du MIT.
- Sedgewick, Wayne – *Algorithms* (4e édition), chapitres 2.2 (Merge Sort) et 4.1 (Divide and Conquer). Manuel du cours Princeton Algorithms.
- MIT OpenCourseWare 6.006 – Introduction to Algorithms (lectures 3-4).
- Stanford CS161 – Design and Analysis of Algorithms (lecture 3 sur Karatsuba et récurrences).
- Harvard CS50 – Introduction to Programming with Python (semaine 6 sur la récursivité).

2 Objectifs de la séance

Objectifs de la séance

- Comprendre le mécanisme de la récursivité et les risques associés (profondeur de pile, terminaison).
- Maîtriser le paradigme **Diviser pour Régner** : décomposition, résolution des sous-problèmes, combinaison.
- Introduire les bases de la **Programmation Orientée Objet** (POO) en Python : classes, attributs, méthodes, constructeur, représentation.
- Implémenter trois algorithmes emblématiques :
 - Exponentiation rapide (récursive et itérative)
 - Multiplication de Karatsuba (pour grands entiers)
 - Tri fusion (merge sort)
- Analyser leur complexité via le **Master Theorem** et la mesurer empiriquement.

3 Introduction à la Programmation Orientée Objet (POO)

3.1 Pourquoi la POO pour un mathématicien ?

La POO permet de regrouper **données** et **comportements** au sein de **classes**, ce qui est particulièrement utile pour modéliser des objets mathématiques (polynômes, matrices, grands

entiers). Comme le souligne le cours *Mathematical Logic through Python* de l'Université de Cambridge, « l'implémentation des concepts logiques et des preuves mathématiques via la programmation Python »[reference :6] passe naturellement par une approche orientée objet.

3.2 Syntaxe de base d'une classe en Python

```

1 class Rationnel:
2     """Repr sente un nombre rationnel num/den."""
3
4     def __init__(self, numérateur, denominateur=1):
5         # Le constructeur est appel la cr ation d'une instance
6         if denominateur == 0:
7             raise ValueError("D nominateur nul")
8         self.num = numérateur
9         self.den = denominateur
10        self._simplifier()
11
12    def _simplifier(self):
13        """M thode priv e pour simplifier la fraction."""
14        from math import gcd
15        d = gcd(self.num, self.den)
16        self.num //= d
17        self.den //= d
18        if self.den < 0:
19            self.num = -self.num
20            self.den = -self.den
21
22    def __str__(self):
23        """Repr sentation informelle (print)."""
24        return f"{self.num}/{self.den}"
25
26    def __repr__(self):
27        """Repr sentation formelle (d bogage)."""
28        return f"Rationnel({self.num}, {self.den})"
29
30    def __add__(self, other):
31        """Surcharge de l'op rateur +."""
32        return Rationnel(self.num * other.den + other.num * self.den,
33                          self.den * other.den)
34
35 # Utilisation
36 r1 = Rationnel(1, 2)    # 1/2
37 r2 = Rationnel(3, 4)    # 3/4
38 print(r1 + r2)          # 5/4

```

Listing 1 – Définition d'une classe simple

Exemple 1 (Encapsulation d'un polynôme). *Créons une classe simple pour représenter un polynôme :*

```

1 class Polynome:
2     def __init__(self, coefficients):
3         # coefficients[0] = terme constant
4         self.coeffs = coefficients[:]
5         self._nettoyer()
6
7     def _nettoyer(self):
8         while len(self.coeffs) > 1 and self.coeffs[-1] == 0:
9             self.coeffs.pop()
10
11    def evaluer(self, x):
12        """ value le polyn me en x (m thode de Horner)."""

```

```

13     resultat = 0
14     for c in reversed(self.coef):
15         resultat = resultat * x + c
16     return resultat
17
18     def __str__(self):
19         terms = []
20         for i, c in enumerate(self.coef):
21             if c != 0:
22                 if i == 0:
23                     terms.append(str(c))
24                 elif i == 1:
25                     terms.append(f"{c}*x")
26                 else:
27                     terms.append(f"{c}*x^{i}")
28         return " + ".join(terms) if terms else "0"
29
30 # Test
31 p = Polynome([1, 2, 3]) # 1 + 2x + 3x^2
32 print(p)               # 1 + 2*x + 3*x^2
33 print(p.evaluer(2))    # 1 + 4 + 12 = 17

```

3.3 Avantages de la POO

- **Encapsulation** : un objet `GrandEntier` peut cacher les détails de stockage (tableau de chiffres) et fournir des méthodes `multiplier` optimisées.
- **Réutilisabilité** : une classe `Polynome` peut être utilisée pour implémenter Karatsuba, puis enrichie pour la dérivation symbolique.
- **Clarté** : le code reflète la structure mathématique sous-jacente.

4 Récursivité : principes et mécanismes

Définition 1 (Fonction récursive). Une fonction *récursive* est une fonction qui s'appelle elle-même. Elle doit comporter :

- Un ou plusieurs *cas de base* (condition d'arrêt).
- Un *appel récursif* qui fait progresser le calcul vers le cas de base.

Exemple 2 (Factorielle récursive). `def factorielle(n):`

```

2     if n == 0: # cas de base
3         return 1
4     else:
5         return n * factorielle(n-1) # appel récursif

```

Déroulement pour $n=3$:

$$\begin{aligned}
 \text{factorielle}(3) &= 3 \times \text{factorielle}(2) \\
 &= 3 \times (2 \times \text{factorielle}(1)) \\
 &= 3 \times (2 \times (1 \times \text{factorielle}(0))) \\
 &= 3 \times (2 \times (1 \times 1)) = 6
 \end{aligned}$$

Exemple 3 (Fibonacci récursif). `def fibonacci(n):`

```

2     if n <= 1:
3         return n
4     return fibonacci(n-1) + fibonacci(n-2)

```

Attention : Cette version naïve a une complexité exponentielle $O(2^n)$! Nous verrons comment l'améliorer par la suite.

Exemple 4 (Somme des éléments d'une liste): `def somme_liste(liste):`

```
2     if not liste:           # cas de base : liste vide
3         return 0
4     return liste[0] + somme_liste(liste[1:]) # premier lment + somme du
    reste
```

4.1 Pièges classiques

Attention

- **Profondeur de récursion** : Python limite la pile d'appels (environ 1000). Au-delà, une `RecursionError` est levée.
- **Récursion terminale** : non optimisée en Python (contrairement à certains langages fonctionnels comme Scheme ou Haskell).
- **Oubli du cas de base** : provoque une récursion infinie.

Exemple 5 (Profondeur de récursion excessive): `def compte_a_rebours(n):`

```
2     print(n)
3     compte_a_rebours(n-1) # Pas de cas de base !
4
5 # compte_a_rebours(10) # Provoque une RecursionError
```

Correction :

```
1 def compte_a_rebours(n):
2     if n < 0:           # cas de base
3         return
4     print(n)
5     compte_a_rebours(n-1)
```

5 Paradigme « Diviser pour Régner »

Définition 2. Un algorithme *Diviser pour Régner* suit trois étapes, telles que définies dans le cours 6.006 du MIT [référence :7] :

1. **Diviser** le problème en plusieurs sous-problèmes de taille plus petite.
2. **Régner** sur les sous-problèmes (généralement de manière récursive).
3. **Combiner** les solutions des sous-problèmes pour obtenir la solution du problème original.

La complexité $T(n)$ d'un tel algorithme satisfait une récurrence du type :

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

où a est le nombre de sous-problèmes, b le facteur de division et $f(n)$ le coût de division/combinaison.

Proposition 1 (Master Theorem – version simplifiée). Soit $T(n) = aT(n/b) + O(n^d)$. Alors :

$$T(n) = \begin{cases} O(n^d) & \text{si } d > \log_b a \\ O(n^d \log n) & \text{si } d = \log_b a \\ O(n^{\log_b a}) & \text{si } d < \log_b a \end{cases}$$

6 Algorithmes à implémenter

6.1 Exponentiation rapide

L'exponentiation naïve de x^n demande $n-1$ multiplications. L'exponentiation rapide (square-and-multiply) exploite la parité de l'exposant[reference :8] :

$$x^n = \begin{cases} (x^{n/2})^2 & \text{si } n \text{ pair} \\ x \cdot (x^{(n-1)/2})^2 & \text{si } n \text{ impair} \end{cases}$$

Complexité : $O(\log n)$ multiplications.

Version récursive

```
1 def puissance_rapide(x, n):
2     if n == 0:
3         return 1
4     elif n % 2 == 0:
5         demi = puissance_rapide(x, n // 2)
6         return demi * demi
7     else:
8         demi = puissance_rapide(x, (n - 1) // 2)
9         return x * demi * demi
```

Listing 2 – Exponentiation rapide récursive

Version itérative (méthode binaire)

```
1 def puissance_rapide_iter(x, n):
2     resultat = 1
3     base = x
4     while n > 0:
5         if n % 2 == 1:
6             resultat *= base
7         base *= base
8         n //= 2
9     return resultat
```

Listing 3 – Exponentiation rapide itérative

Encapsulation dans une classe

```
1 class Exponentiateur:
2     def __init__(self, base):
3         self.base = base
4
5     def puissance_rec(self, n):
6         if n == 0:
7             return 1
8         demi = self.puissance_rec(n // 2)
9         if n % 2 == 0:
10            return demi * demi
11        else:
12            return self.base * demi * demi
13
14    def puissance_iter(self, n):
15        res = 1
16        b = self.base
17        exp = n
```

```

18     while exp > 0:
19         if exp % 2 == 1:
20             res *= b
21         b *= b
22         exp //= 2
23     return res

```

6.2 Multiplication de Karatsuba

L'algorithme de Karatsuba (1960), étudié en détail dans Stanford CS161[reference :9], améliore la multiplication de deux entiers de n chiffres de $O(n^2)$ à $O(n^{\log_2 3}) \approx O(n^{1.585})$.

Soient x et y deux entiers. On écrit :

$$x = x_1 \cdot B^m + x_0, \quad y = y_1 \cdot B^m + y_0$$

avec $m = \lceil n/2 \rceil$ et $B = 10$ (ou 2 en binaire). Alors :

$$x \cdot y = x_1 y_1 B^{2m} + ((x_1 + x_0)(y_1 + y_0) - x_1 y_1 - x_0 y_0) B^m + x_0 y_0$$

On passe de 4 multiplications à 3.

Implémentation avec des entiers Python

```

1 def karatsuba(x, y):
2     if x < 10 or y < 10:
3         return x * y
4
5     # Calcul de la taille
6     n = max(len(str(x)), len(str(y)))
7     m = n // 2
8
9     # Division
10    Bm = 10**m
11    x1, x0 = divmod(x, Bm)
12    y1, y0 = divmod(y, Bm)
13
14    # Appels récursifs
15    z0 = karatsuba(x0, y0)
16    z2 = karatsuba(x1, y1)
17    z1 = karatsuba(x0 + x1, y0 + y1) - z2 - z0
18
19    return (z2 * 10**(2*m)) + (z1 * 10**m) + z0

```

Listing 4 – Karatsuba récursif

Version orientée objet : classe GrandEntier

```

1 class GrandEntier:
2     def __init__(self, valeur):
3         self.val = int(valeur)
4
5     def __mul__(self, other):
6         if isinstance(other, GrandEntier):
7             return GrandEntier(karatsuba(self.val, other.val))
8         else:
9             return GrandEntier(karatsuba(self.val, other))
10
11    def __rmul__(self, other):
12        return self.__mul__(other)

```

```

13
14     def __repr__(self):
15         return f"GrandEntier({self.val})"
16
17     def __str__(self):
18         return str(self.val)

```

6.3 Tri fusion (Merge Sort)

Le tri fusion est un exemple classique de « diviser pour régner », étudié en profondeur dans le cours Algorithms de Princeton[reference :10][reference :11].

- **Diviser** : couper la liste en deux moitiés.
- **Régner** : trier récursivement chaque moitié.
- **Combiner** : fusionner les deux listes triées.

Complexité : $O(n \log n)$.

```

1 def fusion(gauche, droite):
2     resultat = []
3     i = j = 0
4     while i < len(gauche) and j < len(droite):
5         if gauche[i] <= droite[j]:
6             resultat.append(gauche[i])
7             i += 1
8         else:
9             resultat.append(droite[j])
10            j += 1
11    resultat.extend(gauche[i:])
12    resultat.extend(droite[j:])
13    return resultat
14
15 def tri_fusion(liste):
16     if len(liste) <= 1:
17         return liste
18     milieu = len(liste) // 2
19     gauche = tri_fusion(liste[:milieu])
20     droite = tri_fusion(liste[milieu:])
21     return fusion(gauche, droite)

```

Listing 5 – Tri fusion

Encapsulation : classe Trieur

```

1 class Trieur:
2     @staticmethod
3     def fusionner(gauche, droite):
4         resultat = []
5         i = j = 0
6         while i < len(gauche) and j < len(droite):
7             if gauche[i] <= droite[j]:
8                 resultat.append(gauche[i])
9                 i += 1
10            else:
11                resultat.append(droite[j])
12                j += 1
13            resultat.extend(gauche[i:])
14            resultat.extend(droite[j:])
15            return resultat
16
17     @classmethod

```



```

18     def trier(cls, liste):
19         if len(liste) <= 1:
20             return liste
21         milieu = len(liste) // 2
22         return cls.fusionner(cls.trier(liste[:milieu]), cls.trier(liste[milieu
:]))

```

7 Exercices

7.1 Exercices sur la récursivité

Exercice 1 (Somme des entiers). Écrivez une fonction récursive `somme_entiers(n)` qui retourne la somme des entiers de 1 à n . Testez avec $n = 10$ (résultat attendu : 55).

Exercice 2 (Puissance de 2). Écrivez une fonction récursive `puissance_de_2(n)` qui retourne 2^n . N'utilisez pas l'opérateur `**`. Testez avec $n = 5$ (résultat attendu : 32).

Exercice 3 (Inversion de chaîne). Écrivez une fonction récursive `inverser(chaîne)` qui retourne la chaîne inversée. Par exemple, `inverser("python")` doit retourner `"nohtyp"`. Inspirez-vous des exercices de récursivité du cours CS50 de Harvard[reference :12].

Exercice 4 (Recherche dichotomique). La recherche dichotomique (binary search) est un algorithme « diviser pour régner ». Implémentez une version récursive qui prend une liste triée et un élément à chercher, et retourne l'indice de l'élément ou -1 s'il n'est pas trouvé.

Exercice 5 (Nombre de chiffres). Écrivez une fonction récursive `nb_chiffres(n)` qui retourne le nombre de chiffres d'un entier positif n . Par exemple, `nb_chiffres(12345)` retourne 5.

7.2 Exercices sur la POO

Exercice 6 (Classe Vecteur2D). Créez une classe `Vecteur2D` avec les attributs `x` et `y`. Implémentez les méthodes `__add__` (addition de deux vecteurs), `norme` (retourne $\sqrt{x^2 + y^2}$) et `__str__`. Testez avec `v1 = Vecteur2D(3, 4)` et `v2 = Vecteur2D(1, 2)`.

Exercice 7 (Classe Pile (Stack)). Implémentez une classe `Pile` qui gère une pile LIFO (Last In, First Out). Elle doit avoir les méthodes `empiler(element)`, `depiler()` et `est_vide()`.

Exercice 8 (Classe Matrice). Créez une classe `Matrice` représentant une matrice $m \times n$. Implémentez `__add__` et `__mul__` (multiplication matricielle). Ajoutez une méthode `transposee()` qui retourne la transposée.

7.3 Exercices sur Diviser pour Régner

Exercice 9 (Maximum d'une liste). Implémentez une fonction récursive `maximum_liste(liste)` qui utilise le paradigme « diviser pour régner » pour trouver l'élément maximum. Comparez la complexité avec une approche itérative.

Exercice 10 (Tour de Hanoï). Le problème des Tours de Hanoï est un classique de la récursivité. Écrivez une fonction `hanoi(n, source, destination, auxiliaire)` qui affiche les déplacements nécessaires pour déplacer n disques. Testez avec $n = 3$.

Exercice 11 (Multiplication de matrices par Strassen). L'algorithme de Strassen (étudié au MIT[reference :13]) est une généralisation de Karatsuba aux matrices. Pour des matrices 2×2 , il utilise 7 multiplications au lieu de 8. Implémentez-le pour des matrices carrées de taille puissance de 2.

Exercice 12 (Plus proche paire de points). Implémentez l'algorithme « diviser pour régner » pour trouver la paire de points la plus proche dans un ensemble de points 2D. Cet exercice est inspiré des problèmes avancés du cours CMU 15-112[reference :14].

8 Analyse de complexité et mesures

8.1 Application du Master Theorem

- **Exponentiation rapide** : $T(n) = T(n/2) + O(1) \Rightarrow O(\log n)$.
- **Karatsuba** : $T(n) = 3T(n/2) + O(n) \Rightarrow O(n^{\log_2 3}) \approx O(n^{1.585})$.
- **Tri fusion** : $T(n) = 2T(n/2) + O(n) \Rightarrow O(n \log n)$.

8.2 Mesures empiriques

Comme lors de la séance 1, utilisez `timeit` pour comparer :

- Multiplication naïve de Python (*) vs Karatsuba pour de très grands entiers (au-delà de 10^{1000}).
- Tri fusion vs `sorted()` (Timsort) pour des listes aléatoires.

9 Livrable étudiant

Livrable étudiant

Pour cette séance, vous enrichirez votre dépôt GitHub **M1-Informatique-S1** avec :

- Un notebook Jupyter **seance2.ipynb** contenant :
 - L'implémentation des trois algorithmes (exponentiation rapide, Karatsuba, tri fusion) sous forme de fonctions ET de classes.
 - Les réponses aux exercices (au moins 5 exercices de votre choix parmi ceux proposés).
 - Des mesures de performance avec `timeit` et des graphiques comparatifs.
 - Une analyse de complexité théorique et empirique.
- Un fichier **README.md** mis à jour décrivant les ajouts.
- Un commit avec le message "Séance 2 : Récursivité, POO et Diviser pour Régner".

Critères d'évaluation (sur 10 points) :

- Correction et élégance du code (POO utilisée à bon escient) (3 pts)
- Implémentation correcte de Karatsuba et du tri fusion (3 pts)
- Réalisation des exercices (au moins 5) et analyse de complexité (2 pts)
- Documentation et utilisation de Git (2 pts)

Bibliographie

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
2. Sedgewick, R., & Wayne, K. *Algorithms*, 4th ed. Addison-Wesley, 2011.
3. MIT OpenCourseWare. *6.006 Introduction to Algorithms*, Lectures 3-4.
4. Stanford University. *CS161 : Design and Analysis of Algorithms*, Lecture 3.
5. Princeton University (Coursera). *Algorithms, Part I*, Week 3 : Mergesort.
6. Harvard University. *CS50 : Introduction to Computer Science*, Week 3 (Algorithms).
7. ETH Zürich. *252-0002-AAL Data Structures and Algorithms*, Spring 2025.

8. Carnegie Mellon University. *15-112 : Fundamentals of Programming*, Week 4 : Recursion.
9. University of Cambridge. *Mathematical Logic through Python*, 2023.